

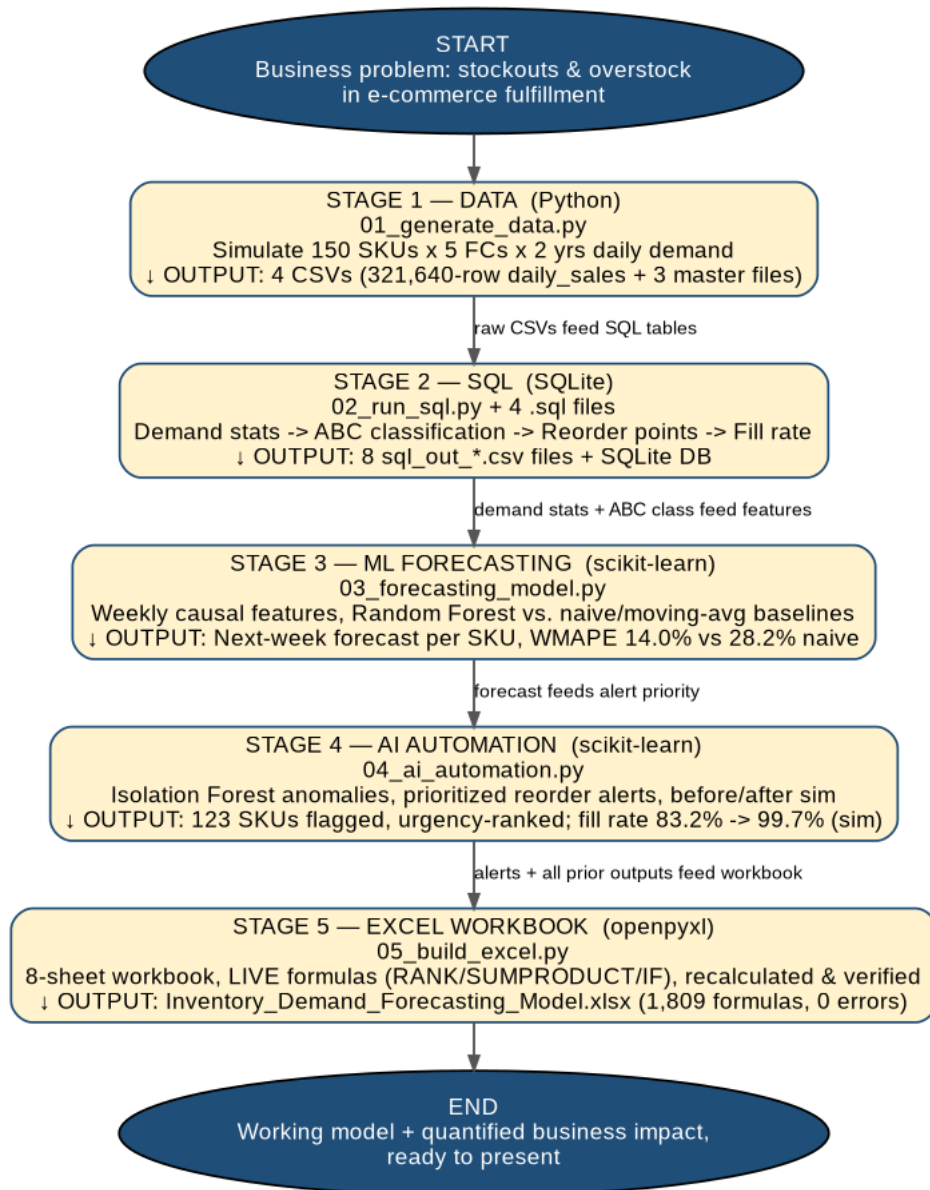
End-to-End Project Flow

Why Each Piece Was Built, How Everything Connects, Step by Step to the Final Output

This document traces the project from raw data to final deliverable, in the exact order it was built. For every step: what went IN, what was DONE, WHY it was done that way, and what came OUT — plus how SQL and Python hand off to each other and to Excel.

1. The Full Pipeline at a Glance

Five stages, each one consuming the previous stage's output as its own input. This is the core idea to say out loud in an interview: nothing in this project is a standalone script — every stage is deliberately chained so the whole thing is one connected pipeline, not five unrelated exercises.



2. Why These Specific Datasets Were Chosen

Before the flow itself, it's worth being able to justify the data design choices — this usually comes up as "why did you structure it this way?"

Dataset	Why this exact structure
sku_master.csv (150 SKUs, 6 categories)	150 is large enough to make ABC analysis and aggregate statistics meaningful (a 10-SKU catalog wouldn't produce a believable Pareto curve), but small enough to stay fast to compute and easy to spot-check by hand. 6 categories spanning very different price points (₹49 grocery items to ₹45,000 electronics) forces the model to handle real heterogeneity, not just one uniform product type.
fulfillment_centers.csv (5 FCs)	A single-warehouse model can't demonstrate network-level thinking (regional availability, multi-location reorder logic) — 5 FCs across 5 real Indian regions (Mumbai, Bengaluru, Delhi NCR, Kolkata, Hyderabad) mirrors a real multi-node fulfillment network like Flipkart's, without the complexity exploding.
daily_sales.csv (321,640 rows, 2 years)	Daily grain is the actual transactional grain real fulfillment systems record at — you cannot compute a trustworthy standard deviation of demand, or train a forecasting model, from weekly or monthly pre-aggregated data; the noise/seasonality has to exist at the daily level first, and get deliberately aggregated up later ONLY where appropriate (ML forecasting). 2 years is enough history to cover multiple festive cycles (Diwali twice) so seasonality is learnable, not a one-off.
inventory_snapshot.csv (current on-hand stock)	Every reorder decision needs a CURRENT stock number to compare against the reorder point — this file represents "what a live inventory system query would return right now," separating the historical fact table (daily_sales) from the operational current-state snapshot, which is how real systems are actually architected (transactional log vs. current-state table).

3. How the Files Relate to Each Other (Data Lineage)

Every file after the first stage exists because an earlier file produced the input it needed. Nothing was created independently.

File	Depends on	Used by
sku_master.csv	— (source of truth)	daily_sales generation, all SQL joins, forecast features, Excel SKU_Master sheet
daily_sales.csv	sku_master.csv (lead time, base demand, assigned FCs)	SQLite DB load, weekly aggregation for ML, anomaly detection
inventory_snapshot.csv	daily_sales.csv (last day's closing stock)	SQL reorder action list, AI automation alerts, Excel reorder calculator
ecommerce_fulfillment.db	all 4 CSVs (loaded as tables)	all 4 .sql files
sql_out_demand_stats.csv	v_sku_demand_stats_90d (SQL view on daily_sales)	sql_out_reorder_points.csv, Excel Demand_Stats sheet
sql_out_abc_classification.csv	v_abc_classification (SQL view, uses sku_master + daily_sales)	sql_out_reorder_points.csv, ai_reorder_alerts.csv, Excel ABC_Analysis sheet
sql_out_reorder_points.csv	demand_stats + abc_classification + sku_master (joined in SQL)	ai_reorder_alerts.csv, ai_before_after_impact.csv, Excel Reorder_Point_Calc sheet

File	Depends on	Used by
forecast_next_week.csv	daily_sales.csv (aggregated to weekly + engineered features)	ai_reorder_alerts.csv (forecast-driven urgency), Excel Forecast_Model sheet
ai_reorder_alerts.csv	sql_out_reorder_points.csv + forecast_next_week.csv + inventory_snapshot.csv	Excel Reorder_Alerts sheet
ai_before_after_impact.csv	sql_out_reorder_points.csv + daily_sales.csv (replayed)	Excel Fill_Rate_Impact sheet
Inventory_Demand_Forecasting_Model.xlsx	every CSV above	final deliverable — nothing downstream

4. How SQL and Python Are Actually Connected

This is the part interviewers most often probe, because it's easy to fake having done both without ever really connecting them. Here's exactly how the handoff works, mechanically:

Direction	Mechanism	Code
Python → SQL	pandas DataFrames are written to SQLite tables using <code>.to_sql()</code> , turning CSV files into real queryable database tables	<code>sku.to_sql('sku_master', conn, if_exists='replace', index=False)</code>
Python → SQL	Python opens each raw <code>.sql</code> file as plain text and executes the entire script against the database connection in one call	<code>conn.executescript(open('sql/03_abc_analysis.sql').read())</code>
SQL → Python	pandas reads a SQL query's result directly back into a DataFrame — this is the actual bridge that lets Python resume working with SQL's output	<code>abc = pd.read_sql('SELECT * FROM v_abc_classification', conn)</code>
Python (SQL output) → Python (ML)	The exported <code>sql_out_*.csv</code> files are read fresh by the forecasting/automation scripts as plain input data — SQL's job is finished by this point	<code>reorder_pts = pd.read_csv('sql_out_reorder_points.csv')</code>
Python (all outputs) → Excel	Every CSV (SQL outputs, ML outputs, AI automation outputs) is read one more time by the Excel-builder script and either written as a formula input or referenced by a live Excel formula	<code>abc_src = pd.read_csv('sql_out_abc_classification.csv')</code>

In one sentence for the interview: "Python generates and orchestrates, SQLite stores and does the set-based aggregation work SQL is best at, and pandas is the bridge that moves data back and forth between the two — nothing is done twice, and each tool does the part it's actually good at."

5. Step-by-Step: Start to Finish

STEP 1 — Generate the Synthetic Dataset

INPUT	Nothing — this is the first step. Only design decisions (150 SKUs, 6 categories, 5 FCs, 2-year window).
WHAT WAS DONE	Ran scripts/01_generate_data.py. Built the SKU catalog and FC list first, then simulated daily demand for every SKU x FC x day using a Poisson distribution, layering in weekend uplift, festive sale multipliers, and a mild growth trend. Ran a deliberately naive weekly-review reorder policy underneath the simulation so realistic stockouts occur.
WHY IT WAS DONE	No real Flipkart-scale dataset is publicly available. A believable synthetic dataset — one built on the right statistical distribution (Poisson for count data) with genuine seasonality and a realistic (imperfect) baseline policy — is what makes every later stage's output defensible and interview-ready, instead of arbitrary.
OUTPUT PRODUCED	4 CSV files: sku_master.csv (150 rows), fulfillment_centers.csv (5 rows), daily_sales.csv (321,640 rows), inventory_snapshot.csv (440 rows). Baseline fill rate measured at 87.6% (later refined to 83.2% for the 12-week comparison window).

STEP 2 — Load Data into SQL and Run the Analytical Queries

INPUT	The 4 CSV files from Step 1.
WHAT WAS DONE	Ran scripts/02_run_sql.py. Loaded all 4 CSVs into a SQLite database as real tables (with indexes on sku_id and date for query speed), then executed the 4 SQL files in sequence: demand statistics first, then ABC classification, then reorder points (which needs both of the first two), then fill rate impact.
WHY IT WAS DONE	SQL is the right tool for set-based aggregation over hundreds of thousands of rows — window functions (RANK, running SUM) and GROUP BY aggregations are exactly what a relational database is optimized for, and it's also the tool the resume bullet explicitly calls for.
OUTPUT PRODUCED	1 SQLite database (ecommerce_fulfillment.db) plus 8 sql_out_*.csv files: demand stats, ABC classification (+ summary), reorder points, reorder actions, fill rate by class, network fill rate, worst-fill-rate SKUs.

STEP 3 — Build the ML Demand Forecasting Model

INPUT	daily_sales.csv (raw) for weekly aggregation, sku_master.csv for category features.
WHAT WAS DONE	Ran scripts/03_forecasting_model.py. Aggregated daily demand to weekly per SKU. Engineered causal lag/rolling/calendar features (nothing from the future). Split train/test by TIME (last 8 weeks held out). Trained and benchmarked 4 approaches: naive, moving average, Random

	Forest, Gradient Boosting.
WHY IT WAS DONE	ABC + reorder points (Steps 1-2) use historical AVERAGES, which don't adapt to trend or upcoming seasonality. A trained forecasting model can react to those patterns — this step exists to prove that ML forecasting genuinely beats simpler statistical approaches, with a fair, leakage-free benchmark, not just to add "AI" for its own sake.
OUTPUT PRODUCED	forecast_model_comparison.csv (WMAPE: naive 28.2% -> Random Forest 14.0%), forecast_feature_importance.csv, forecast_next_week.csv (150-row forecast, one per SKU), forecast_test_predictions.csv.

STEP 4 — Build the AI Automation Layer

INPUT	sql_out_reorder_points.csv (Step 2), forecast_next_week.csv (Step 3), inventory_snapshot.csv (Step 1), daily_sales.csv (Step 1, for the replay simulation).
WHAT WAS DONE	Ran scripts/04_ai_automation.py. (a) Ran Isolation Forest on each SKU's demand-vs-trend ratio to flag anomalous weeks. (b) Joined reorder points + forecast + current stock into a prioritized, human-readable alert list (CRITICAL/HIGH/MEDIUM). (c) Replayed the last 12 weeks of REAL demand through a simulated new policy (topped-up stock, event-driven reordering) to estimate fill-rate impact.
WHY IT WAS DONE	A model that only calculates numbers isn't automation — this step turns the calculations from Steps 2-3 into things a real operations team could act on immediately (a prioritized to-do list) and proves the business impact with a like-for-like before/after comparison, not just a theoretical claim.
OUTPUT PRODUCED	ai_demand_anomalies.csv (1,050 flagged SKU-weeks), ai_reorder_alerts.csv (123 of 150 SKUs flagged, urgency-ranked), ai_before_after_impact.csv (83.2% -> 99.7% fill rate).

STEP 5 — Build the Excel Workbook

INPUT	Every CSV produced by Steps 1-4.
WHAT WAS DONE	Ran scripts/05_build_excel.py. Built an 8-sheet workbook: README, SKU_Master, Demand_Stats, ABC_Analysis (live formulas), Reorder_Point_Calc (live formulas, cross-sheet references), Fill_Rate_Impact (with chart), Forecast_Model, Reorder_Alerts (conditional formatting). Then ran a LibreOffice recalculation pass to force every formula to compute and check for errors.
WHY IT WAS DONE	The resume bullet specifically says "Excel" — a workbook of pasted static numbers wouldn't actually demonstrate Excel modeling skill. Building the ABC classification and reorder-point calculation as LIVE formulas (RANK, SUMPRODUCT, IF, cross-sheet references) makes it a real interactive model an analyst could open, audit, and recalculate — not just a report shaped like a spreadsheet.

OUTPUT PRODUCED	Inventory_Demand_Forecasting_Model.xlsx — 1,809 formulas, 0 errors on recalculation, verified against the SQL layer's numbers (e.g. ABC split 60/44/46 SKUs matches exactly).

End state

A working, auditable pipeline where every number traces back to a specific script and a specific formula — and a quantified result: forecast error cut roughly in half (28.2% -> 14.0% WMAPE), and a simulated fill-rate improvement of +16.5 percentage points (83.2% -> 99.7%), stated alongside the honest caveat that it's a backtest, not a live A/B test.